

Project Proposal

Data Structures

City Route Finder

Group Members:

Danish Ahmed (24k-3025) (Group Leader)

Muneer Ahmed (21i-1179)

Abdul Rafay (24k-3061)

Introduction:

The *City Route Finder* is a console-based application that helps users find the shortest and most efficient route between two cities. The project is based on the concept of **graphs**, where each city is represented as a node and each road connecting two cities is represented as an edge with a specific distance.

The program allows users to input a source and destination city and then applies efficient **graph traversal algorithms** like **Dijkstra's Algorithm** or **Breadth-First Search (BFS)** to compute the shortest path.

This project demonstrates how **data structures and algorithms** can be applied to solve real-world navigation and path-finding problems.

Problem Statement:

In real life, people and transport systems depend heavily on maps and navigation systems (like Google Maps) to find optimal routes. Developing a City Route Finder allows us to explore how these systems work internally — through **graphs, priority queues, and shortest path algorithms**.

This project is both **educational and practical**, as it connects DSA theory with a real-world use case. It will also enhance our understanding of:

- Graph representations
- Shortest path algorithms
- Data structure optimization for performance

Real-World Use Cases:

1. Navigation Systems:

Apps like Google Maps, Uber, and Careem use similar algorithms to find shortest driving routes between two locations.

2. Logistics & Delivery Planning:

Courier and delivery services (like TCS or Foodpanda) can use such algorithms to find the most efficient delivery paths to save time and fuel.

3. Disaster Management:

Government agencies can use route finders to plan evacuation or rescue routes efficiently during floods or earthquakes.

Usage of Data Structures:

Data Structure	Where It's Used	Why It's Used (Justification)
1. Graph (Adjacency List)	To represent cities and the roads connecting them.	Efficient for storing sparse connections between cities and performing graph algorithms.
2. Priority Queue (Min-Heap)	Used in Dijkstra's Algorithm to always pick the next city with the smallest distance.	Provides optimal time complexity for selecting the minimum distance node.
3. Stack	To reconstruct and print the final route (from destination back to source).	LIFO structure helps reverse the path easily.
4. Queue	Used in BFS for exploring routes level-by-level.	Ensures breadth-wise traversal, suitable for unweighted graphs.
5. Map (Hash Map / Ordered Map)	To store city names, distances, and parent relationships.	Allows fast lookup of city data and route mapping.

Features:

- Add and display connected cities and distances.
- Take input for source and destination cities.
- Display the shortest path and total distance.
- Handle invalid inputs gracefully.
- Option to view all city connections.

Tools and Technologies

- **Programming Language:** C++
- **IDE:** Visual Studio Code
- **Libraries Used:** Standard Template Library (STL) for map, vector, queue, priority_queue, and stack.